Where Does the Probability Cost Live? A Distributional Test of Whether CUDA Tokens Approach the Claimed Probability Floor

Author: leo-grayrat **Affiliation:** Independent research **Date:** 2026-10-17

Abstract

CUDA Agent explains its policy collapse at training step 17 with a chain whose second link is an explicit empirical claim: CUDA code is under 0.01% of pretraining data, therefore the policy assigns near-floor probability to CUDA tokens, "e.g. $\pi_\theta(a_t \mid s_t) \approx 10^{-9}$," and small precision differences are consequently amplified into exploding importance ratios. We test that link where it can be tested: at generation time, against a live model, with per-token conditional probabilities.

Two collections support the test. The first uses four elementwise/reduction tasks (128 runs, 98,533 tokens). The second uses five KernelBench Level 2 operator chains — ConvTranspose3d + GroupNorm + GELU + Sum and similar — where an implementer must decide what to fuse, whether to materialise intermediates, and how to tile (40 runs, 542,611 tokens; **87.5% of these tokens come from runs that ended in failure**, a selection effect noted wherever their numbers are quoted). In both, the kernel is written in the reply body rather than through tool calls, so every code token falls under `logprobs`; a pre-flight check confirmed 448/448 code tokens carry a probability.

Four results. **First**, CUDA/Triton vocabulary carries a *lower* low-probability rate than the surrounding text — 0.26% vs 0.84% under a strict vocabulary (Fisher $p = 0.0007$), 0.49% vs 0.84% under a loose one ($p = 0.0070$). **Second**, the cost is spatially concentrated: inside fenced code blocks the rate is 0.60% (0.33% with in-block comments excluded), outside it 13.56% — a 22.4× difference that rises to 41× once comments are removed from the code region. The worst 5% of code tokens carry 98.1% of the code's total negative log-probability while the worst 5% of prose tokens carry only 30.3% of prose's. **Third**, raising task difficulty does not move the code-side rate: 0.67% on reduction tasks, 0.60% on fusion tasks, with per-task values spanning 0.25–0.93%. **Fourth**, the absolute probabilities do not approach the claimed floor: the lowest code token in 322,375 measured has logprob −13.02 (2.2×10), the lowest prose token −16.65 (5.9×10), while 10 corresponds to −20.7 — 400× below anything observed.

Reading all 816 low-probability tokens from the first collection and all 189 high-cost non-comment code tokens from the second, in context and without a keyword table, yields a consistent picture: the cost sits in comments, identifier naming, punctuation and occasional slips, not in implementation choices. We find no case in which the model's uncertainty at a token corresponds to choosing between two ways of implementing a kernel.

We read this as evidence that **whatever the reported 10 measures, it is not a property of the domain's vocabulary** — alignment cannot change corpus statistics, so a rarity-driven floor would survive alignment if it existed. The causal chain's attribution therefore points at the wrong object. The uncertainty tracks *what the model is writing* (prose or debug reasoning vs templated code), not *which vocabulary the domain uses*. The gap between 10 and our measurements is not presented as a refutation, since the source measures a base model during rollout and we measure an aligned model at inference; it is presented as a tension the source must explain, with the resolving experiment named.

Keywords: CUDA Agent; training instability; conditional probability; log-probability distribution;

fusion kernels

---

1. Introduction

1.1 The claim, quoted

Agentic reinforcement learning now produces GPU kernels that beat compiler autotuning. CUDA Agent reports pass rates of 100%, 100% and 92% across the KernelBench difficulty splits and faster-than-`torch.compile` rates of 97%, 97% and 92% ("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026, Tab. 2). It reaches there by running a language model inside a development sandbox and optimising a policy over multi-turn trajectories with PPO.

Its account of the training recipe names one failure and one fix. The failure: "the first run could only train stably for 17 steps before the model's performance collapsed." The diagnosis is distributional — CUDA coding data "accounts for less than 0.01% of pretraining data," producing a "severe domain distribution mismatch" in which the policy must put mass on very rare token sequences "sometimes at probabilities near the numerical floor, e.g. $\pi_\theta(a_t \mid s_t) \approx 10^{-9}$" ("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026, Sec. 5.1–5.2; "CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation" 2026). The consequence is that "even small train-rollout precision differences can explode the importance ratio $\rho_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$." The fix is a three-stage warm-up: single-turn RL, rejection fine-tuning of the actor on positive-reward trajectories, and value pretraining of the critic.

1.2 Why the second link is the one to test

The chain has four links: (i) CUDA is rare in pretraining; (ii) that rarity yields near-floor token probabilities; (iii) near-floor probabilities amplify precision differences into importance-ratio variance; (iv) the ratio explodes and training collapses.

Links (iii) and (iv) require training runs and are outside our reach. But **link (ii) is a statement about a probability distribution at generation time**, and that can be measured directly, on any model, without training. If (ii) fails, (iii) and (iv) have no antecedent, and the attribution needs a different basis.

There is a second reason to test (ii) rather than (iii)/(iv): (ii) is *cheap to falsify and expensive to defend*. Anyone can measure a conditional probability; the claim that it sits at 10 is falsifiable with a `logprobs` request. That asymmetry makes it the right place to push.

1.3 Why the measurement is not trivial

Under an agent framework, the kernel is written through tool calls, and **tool-call arguments are not covered by `logprobs`**. The pattern is consistent across vendors: OpenAI returns `ChoiceLogprobs(content=None)` when `functions` is set (forum threads from 2024 through 2025, no official answer); Claude, Gemini and Grok accept the parameter and silently return none; several reasoning models spend the entire token budget inside the reasoning channel and return empty content, which is a trap we hit ourselves (§3.5). Three papers that examine closed models as scientific instruments converge on the same obstacle and identify it as a policy choice motivated by distillation concerns [8, 9, 10].

The practical consequence is architectural (§3.1): an agent-framework experiment cannot measure the tokens that matter, so the framework has to be removed.

1.4 What this paper does

We measure per-token conditional probabilities inside a real multi-turn kernel-writing loop, on two task families of different difficulty, and ask:

- **RQ1** — Is CUDA/Triton vocabulary more likely to be low-probability than other tokens?
- **RQ2** — Does the low-probability rate vary with *where* the token sits (fenced code vs prose)?
- **RQ3** — Does raising task difficulty from reduction to fusion move the code-side rate?
- **RQ4** — Do the measured absolute probabilities approach the claimed 10 floor?

1.5 Findings

Question	Answer
RQ1	No — CUDA vocabulary is *less* likely to be low-probability (0.26% vs 0.84%, $p = 0.0007$)
RQ2	Yes, by an order of magnitude: code 0.60% vs prose 13.56%
RQ3	No — 0.67% on reductions, 0.60% on fusion chains
RQ4	No — minimum code logprob −13.02 (2.2×10) vs the claimed −20.7

1.6 Scope, stated up front

We measure an **aligned commercial model at inference**, not the un-finetuned base model during rollout. That difference is material and is treated at length in §5.2. It is the reason this paper reports a tension rather than a refutation.

We also do not claim the code-side rate is zero. It is 0.60%, not 0. What we claim is that it does not move with difficulty and does not concentrate at implementation decisions.

---

2. Related Work

2.1 The causal chain under test

CUDA Agent's mechanism, quoted in §1.1, has the shape: rarity → low probability → precision amplification → ratio explosion ("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026, Sec. 5.1). The remedy is staged warm-up with rejection fine-tuning (keep only positive-reward trajectories, discard looping or malformed tool use) and value pretraining with GAE targets ($\gamma = 1$, $\lambda = 0.95$), after which PPO runs with asymmetric clipping ($\varepsilon_{lower} = 0.2$, $\varepsilon_{higher} = 0.28$) for 150 steps ("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026; "CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation" 2026).

An independent reading of the paper is worth quoting here. A third-party overview notes that the authors describe the instability as attributable to the base model's distribution mismatch, and then states plainly: " RFT … " — the paper reports two observed associations (reward collapse with entropy rise when RFT is removed; lower explained variance with longer trajectories when value pretraining is removed), and these are associations plus the authors' interpretation, not proof of a single cause ("Log Probability Tracking of LLM APIs" 2026).

That distinction matters for our purpose. **What was observed is entropy rising; what was claimed is a specific token-level probability near 10 .** These are different objects: entropy is a property of a whole distribution flattening, a near-floor token probability is a point in that distribution. The second does not follow from the first, and the first does not require the second.

2.2 DR. KERNEL: the same instability, attributed to mismatch

DR. KERNEL studies RL for Triton kernel generation and reaches the same instability from the training side ("DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations" 2026). It states two hypotheses — saturation from training instability caused by train/inference mismatch, and an objective misaligned with meaningful speedup — monitors entropy, gradient norms and perplexity, finds "excessively high values across these metrics," and applies geometric Mismatch Rejection Sampling: keep samples whose geometric-mean importance ratio $w = \exp(\frac{1}{|T|}\sum_t \log \frac{\pi_{train}(a_t|s_t)}{\pi_{rollout}(a_t|s_t)})$ lies in $[0.999, 1.001]$, plus a token-level veto rejecting any sequence containing a token whose likelihood ratio falls below 10^{-4} ("DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations" 2026, Sec. 5.1).

Its key negative finding: MRS "successfully stabilizes the training dynamics," but "it does not fundamentally lift the performance ceiling" ("DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations" 2026, Sec. 5.1). Stability and capability are separable, and the thing MRS corrects is a *distributional* discrepancy between two engines — a property of the trajectory's context, not of the vocabulary.

2.3 Precision mismatch and off-policy drift

Jet-RL, *Precision Mismatch in Reasoning Models* and *First-Order Approximation for Stable LLM-RL Training* all concern the sensitivity of the PPO ratio when probabilities sit near the numerical floor; once π_θ is at 10^{-9}, a small absolute difference is a large relative one ("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026; "CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation" 2026). Liu et al. characterise the complementary phenomenon where an RL framework's efficiency machinery makes training data off-policy (Liu and others 2025).

These papers assume the floor condition. Our contribution is to check whether it holds at generation time — a precondition, not a competing mechanism.

2.4 Why closed APIs cannot answer this

See §1.3. The short version: tool-call arguments carry no logprob, so a framework-based measurement cannot see the tokens that write the kernel.

2.5 Position

The works above study instability from the training side. This paper does not train. It measures the generation-time distribution that the causal chain's second link asserts, on a model whose weights we do not control but whose outputs we can read — and it measures it twice, on two task families, with the observable stated as a distribution rather than a threshold.

---

3. Method

3.1 Removing the framework so the measurement exists

An agent writes the kernel through `write` or `edit` tool calls; the kernel text lives in the tool arguments; those tokens carry no logprob (§1.3). Two workarounds fail. Asking the model, via prompt suffix, to first print the code and then call the tool produced compliance in 0 of 6 trials — the model's policy is to act. Replaying saved conversations afterwards cannot recover $\pi_\theta(a_t|s_t)$ either, because the chat template and special tokens applied at generation are not reproducible from a plain reconstruction.

So the tools are removed. The driver sends a task description plus `reference.py` and asks for the complete `candidate.py` inside a fenced block, with no tools offered. The model writes code in `message.content`; that field is covered by `logprobs`. The driver parses the block, writes the file, invokes the independent evaluator, and appends the evaluator's output as the next user turn.

This is the design decision the whole study rests on, so it was verified before collecting: a single-turn probe returned a 1,396-character block containing `@triton.jit` and `tl.load`, with **448 of 448** tokens inside that block carrying a logprob.

3.2 The loop

At turn t the driver holds message list M_t, requests completions with `logprobs=true, top_logprobs=20, temperature=1.0`, receives y_t and $\log \pi_\theta(y_{t,i} \mid M_t, y_{t,<i})$, extracts the fenced block, stages it with `reference.py` and the evaluator in an isolated temp directory, runs the evaluator, and forms feedback f_t:

$$M_{t+1} = M_t \oplus [\text{assistant} : y_t] \oplus [\text{user} : f_t]$$

Termination: correctness after at least one revision, no fenced block, or a turn cap of 6 (level 2) / 8 (reductions).

3.3 Two task families

Family A — reductions (n=4). `row_reduce`, `vector_affine`, `layernorm_rows`, `softmax_rows`. Each is elementwise or a single-dimension reduction; the kernel logic is two or three lines and the control flow is one bounds check.

Family B — operator chains (n=5). From KernelBench Level 2 ("KernelBench: Can LLMs Write Efficient GPU Kernels?" 2025):

Task	Chain
`12_gemm_multiply_leakyrelu`	Linear → scale → LeakyReLU
`1_conv2d_relu_biasadd`	Conv2d → ReLU → bias
`14_gemm_divide_sum_scaling`	Gemm → divide → sum → scale
`17_conv2d_instancenorm_divide`	Conv2d → InstanceNorm → divide
`19_convtranspose2d_gelu_groupnorm`	ConvTranspose2d → GELU → GroupNorm

Each chain admits at least three implementation strategies: fuse everything into one kernel; fuse the elementwise tail and keep the convolution separate; or implement the sequence as written. Choosing among them requires deciding what to fuse, whether intermediates go to global memory, and how to tile. This is exactly the decision space that the reduction tasks lack.

Family B tasks were generated mechanically (`scripts/make_kb2_tasks.py`): the KernelBench `Model` class is wrapped into the `run(x)` entry point, the model is seeded so that reference and candidate see identical weights, and the model is exported as `MODEL` for the candidate to import. Evaluators check correctness on two shapes at tolerance 10^{-4}.

3.4 Feedback conditions

- **full** — the evaluator's verbatim output.
- **truncated** — one line: `compilation failed`, `output mismatch`, or `correct`.

Fixed before collection, not varied afterwards.

3.5 Model selection, settled empirically

The API offers reasoning and non-reasoning models. Under the reasoning model the budget is consumed inside the reasoning channel: at `max_tokens=4000` the probe produced 14,042 characters of reasoning and zero of code, `finish_reason=length`. The non-reasoning model puts everything in `content`, reports zero reasoning tokens, and solved `row_reduce` in two turns in 22 seconds. All reported runs use the non-reasoning model with `max_tokens=8192`.

3.6 Observables

We report the distribution, not only a threshold.

Low-probability rate at threshold τ:

$$R_\tau(G) = \frac{1}{|G|} \sum_{i \in G} \mathbb{1}\left[\log \pi_\theta(a_i \mid s_i) \leq \tau\right]$$

with $\tau = -2$ as the headline. **Quantiles** of the logprob distribution per region. **Minimum logprob** per region, converted to absolute probability for comparison against 10 . **Probability-mass concentration**: what fraction of a region's total negative logprob is carried by its worst 0.1%, 1%, 5% of tokens.

Region assignment is by fenced-code-block state: a token is in `code` if an odd number of "`delimiters precede it, else`prose". No keyword tables are used for the primary results.

3.7 Statistics

Fisher's exact test (two-sided) on the 2×2 table of (low-probability, not) $\times$ (in group, not). Exact rather than asymptotic because cell counts are unbalanced. **The test assumes independent observations, and tokens within a completion are not independent**, so the reported p-values are anti-conservative; the effect sizes here are large enough that the sign is not in question, but $p = 0.0007$ should not be read as a statement about 322,375 independent draws. A cluster-level analysis with run as the unit is not reported.

3.8 Scale

Family	Runs	Tokens
A (reductions)	128	98,533
B (chains)	40	542,611

Family B success rate is 7/40 (17.5%), and its `after_failure` context carries 87.5% of the tokens — a selection effect discussed in §6.

---

4. Results

4.1 RQ1: CUDA vocabulary is not the low-probability tail

Family A, 98,533 content tokens:

Boundary	Group	n	mean logprob	R_{-2}	Fisher p
strict	CUDA/Triton	2,344	−0.013	**0.26%**	**0.0007**
strict	other	96,189	−0.062	0.84%	
loose	CUDA/Triton	4,524	−0.038	**0.49%**	**0.0070**
loose	other	94,009	−0.062	0.84%	

Both boundaries agree in direction, and the direction is opposite to the hypothesis. A third reference class sharpens it: structural tokens (`for`, `if`, `def`, `return`, `in`, `not`) show 1.54% against 0.81% — tokens with *no* choice are more often low-probability than tokens with a domain-specific name.

4.2 RQ2: the cost is spatial, and it is outside the code

Family B, 542,611 tokens, split by fenced-block state:

Task	code tokens	code R_{-2}	prose tokens	prose R_{-2}
`12_gemm_multiply_leakyrelu`	55,419	**0.25%**	25,717	12.62%
`14_gemm_divide_sum_scaling`	46,002	**0.93%**	49,724	14.26%
`17_conv2d_instancenorm_divide`	77,984	**0.58%**	34,586	13.49%
`19_convtranspose2d_gelu_groupnorm`	89,908	**0.73%**	64,942	14.48%
`1_conv2d_relu_biasadd`	53,062	**0.52%**	45,267	12.03%

Task	code tokens	code R_{-2}	prose tokens	prose R_{-2}
total	**322,375**	**0.60%**	**220,236**	**13.56%**

Density ratio: 22.4×.

Comments inside code fences count as code in the table above, and they are high-entropy. Recomputing with in-block comments excluded:

Task	all code	pure code
`12_gemm_multiply_leakyrelu`	0.25%	**0.18%**
`14_gemm_divide_sum_scaling`	0.93%	**0.44%**
`17_conv2d_instancenorm_divide`	0.58%	**0.26%**
`19_convtranspose2d_gelu_groupnorm`	0.73%	**0.42%**
`1_conv2d_relu_biasadd`	0.52%	**0.37%**
total (310,935 tokens)	0.60%	**0.33%**

With comments removed the contrast against prose is 41×, not 22.4×. The earlier figure was a blend.

The highest-frequency low-probability tokens in the whole collection are `the` (118), `` ` `` (116), `,` (70), `.` (64), `and` (63), `I` (55) — prose function words and punctuation.

4.3 RQ3: raising difficulty does not move the code-side rate

Comparing the two families directly is confounded: they differ in task construction, turn cap (8 vs 6), success rate (near 100% vs 17.5%) and context composition, so an equality between their rates could be a genuine invariance or the cancellation of several opposing effects.

The cleaner comparison is within Family B, where the five tasks differ in chain length and in how often the model succeeded (5/8, 1/8, 1/8, 0/8, 0/8):

Task	chain	success	pure-code R_{-2}
`12_gemm_multiply_leakyrelu`	3 ops	5/8	0.18%
`1_conv2d_relu_biasadd`	3 ops	0/8	0.37%
`14_gemm_divide_sum_scaling`	4 ops	1/8	0.44%
`17_conv2d_instancenorm_divide`	3 ops	1/8	0.26%
`19_convtranspose2d_gelu_groupnorm`	3 ops	0/8	0.42%

The rate spans 0.18%–0.44% across a success range from 0/8 to 5/8. **Within this family, neither chain length nor success rate predicts the code-side density.** Across families, the reduction tasks sit at 0.67% and the chains at 0.33% (pure code), which is a *decrease* — not the increase a difficulty hypothesis would predict, though the four confounds above prevent reading that as an effect.

What rose is the prose share of the stream (from negligible in Family A to 40.6% in Family B) and prose's own rate (12–14%).

4.4 RQ4: the absolute probabilities do not approach the claimed floor

Family B, quantiles of logprob by region:

quantile	code	prose
p0.1	−4.460	−9.915
p1.0	−1.361	−5.842
p5.0	−0.063	−3.433
p50.0	−0.000	−0.240
min	**−13.016**	**−16.647**

Converting to absolute probability:

```
code min   -13.02  →  2.2 × 10
prose min  -16.65  →  5.9 × 10
claimed    10         logprob -20.7
```

The lowest code token in 322,375 is 2.2×10 , four hundred times more probable than the claimed floor. 99% of code tokens sit above 0.26, and the median is 1.0.

For completeness, the joint logprob of a whole completion (n=215) has mean −903.3 and median −414.8, i.e. $e^{-414.8} \approx 7 \times 10^{-181}$. That number is far *below* 10 , but it is the product of two thousand tokens, not a single token's probability — the two quantities are not the same object, and the source's quotation is explicitly of the latter.

4.5 Where the probability mass sits

region	tokens	Σ logprob	worst 0.1%	worst 1%	worst 5%
code	322,375	−14,276	13.4%	60.9%	**98.1%**
prose	220,236	−179,942	1.4%	9.3%	**30.3%**

The shapes are opposite. **Code's cost is concentrated**: 5% of code tokens carry 98.1% of the code's total cost; the other 95% are essentially free. **Prose's cost is spread**: the worst 5% carry only 30.3%, and the remaining 95% each contribute a little.

In absolute terms, prose carries **12.6× the total negative logprob of code on 68% as many tokens** — a per-token cost 18× higher.

4.6 Context

context	tokens	R_{-2}
initial	55,736	1.88%
after_success	12,147	4.26%
after_failure	474,728	**6.37%**

Family A showed the same ordering by task, and `after_failure` ran 4.6× `initial`. The direction is consistent across both families: a failed evaluation raises the rate, a successful one does not.

Note the composition: 87.5% of Family B's tokens sit in `after_failure`. This is a selection effect (§6.5).

4.7 Reading the tokens, not counting them

Aggregate rates say where cost is; they do not say what the model was doing. So we read every low-probability token in context.

Family A: all 816 tokens with R_{-2}. Manual classification, no keyword table:

category	share	examples
code comments / docstrings	~75%	`backed`, `moderate`, `overflow`, `Tunables`, `divisible`
identifier naming	~12%	`y`, `acc`, `denom`, `row_ptr`, `orig_shape`
punctuation / whitespace	~10%	`\n\n`, `:\n`, `,`, `(`
conversational prose	~4%	`If`, `Odd`, `Summary`, `verified`
control-flow decisions	**~1%**	`if not x.is_contiguous()`, `for col_start in range(...)`

Family B: all 189 non-comment code tokens with logprob −4.5.

category	count	examples
identifiers (variables, params, function names)	~110	`mean_ptr`, `coff`, `KHW`, `k_rem`, `dots`, `gemv_kernel`
punctuation / whitespace	~40	`)`, `\n`, `,`, `:`
comment residue (missed by the `#` rule)	~15	`where y_in = y_out - ky`, `← NO`, `lsb-for-tie`
numbers / constants	~8	`2048`, `512`, `32`
method calls	~8	`.view`, `.to`, `.where`
operators	~6	`+=`, `-`, `*`
syntactic choices	**~5**	`for kh in static:`, `for i in tl.static_range(0,1):` `pass`, `assert ref is not None`

Not one token, in either collection, corresponds to choosing between two ways of implementing the kernel. No position of the form "fuse or don't fuse," "materialise or keep in registers," "this precision or that" appears among the low-probability set.

The most extreme values illustrate the character of the tail. The single lowest code token is `' //'` at −13.02, inside `tl.arange(0, BLOCK // 1)` — an expression nobody writes on purpose. The lowest prose token is `'|='` at −16.65, inside `` `allow_tf32|=|True` `` — where `=` was intended. Others in the extreme set: `' bi'` in `biasp_ptr`, `' rt'` in `rtne`, `' tir'` in `tirual`, `'?,'` in `atol=?, rtol=?` (placeholder left unreplaced).

These are slips and placeholder artefacts, not design uncertainty.

A second example makes the distinction concrete: the identifier `y` in `y = torch.empty_like(x)` is repeatedly low-probability across runs, because nothing in the context determines whether the output should be called `y` or `out`. The *implementation* at that position — allocate an output tensor — is not in question.

---

5. Discussion

5.1 The second link does not hold

RQ1–RQ4 converge: CUDA vocabulary carries less low-probability mass than the surrounding text; the cost is spatially located outside code blocks; it does not respond to task difficulty; and its absolute magnitude is 400× above the claimed floor even at the extreme.

The claim "CUDA is rare in pretraining, therefore CUDA tokens sit at 10 " does not survive this measurement **in any condition we can observe**.

What follows is narrower and, we think, harder to escape: **alignment cannot change what is in a pretraining corpus.** If a near-floor probability were a consequence of corpus rarity, it would persist across alignment, because the model would still be the same model with the same exposure frequency. We measure 1.0 on an aligned model. Either the alignment step moved a token probability by nine orders of magnitude — which would mean the quantity is not a function of the corpus at all — or the 10 was never a corpus consequence. Both readings leave the attribution without its stated mechanism.

5.2 The nine-orders-of-magnitude gap, and why it is a tension

We measure 1.0 where the source reports 10 . We decline to read that as a refutation, because conditions differ: the source measures an un-finetuned base model during training rollout; we measure an aligned model through an inference API. Alignment raises the probability of common code patterns substantially.

But the difference must not be used to *dismiss* the result either. The claim, as stated, is a consequence of corpus statistics — CUDA is 0.01% of pretraining — and corpus statistics do not depend on which organisation trained the model or whether it was later aligned. **If rarity implied a near-floor conditional probability, it would do so for any model pretrained on comparable data**; a nine-order-of-magnitude difference between two such models is better explained by the claim being false than by one model having a qualitatively different corpus.

The control that would settle it: repeat the measurement on a base model, before fine-tuning. If CUDA token probabilities there are near the floor and rise after alignment, both accounts hold and the chain is intact at the base-model stage. If they are not near the floor even before alignment, the chain fails at its second link. We could not run this control — it requires base-model weights — and it is the single highest-value follow-up.

5.3 What the data do support: the cost is in the writing, not the code

The strongest regularity in both collections is not about domain vocabulary; it is about **which kind of text the model is producing**.

Code blocks: 0.60%. Prose: 13.56%. The ratio is stable across five very different tasks, and it survives a five-fold increase in implementation complexity.

The mechanism is visible in the token inventory. In code, the model writes from templates — `tl.load(x_ptr + offs, mask=mask, other=0.0)` has a nearly determined continuation, and 95% of code tokens are effectively free. In prose, it composes — "let me reconsider whether the stride is wrong" has an enormous continuation space, and the cost spreads over most tokens.

And the prose is concentrated where the model is failing. `after_failure` carries 87.5% of the tokens at 6.37%, against 1.88% at the first turn. When a kernel does not work, the model writes substantially more text, and that text is where the uncertainty lives.

5.4 An alternative account of the training collapse

What follows is a hypothesis, not a finding. We state it first so that the three observations below are read as things it is consistent with, not as things that support it.

Put the pieces together: the source reports entropy rising alongside reward collapse; DR. KERNEL reports that correcting train/inference mismatch stabilises training without raising the ceiling; and our data show uncertainty concentrated in failure-driven prose, not in code.

A coherent reading is that **the instability is a context-distribution problem, not a vocabulary problem**. After a failed evaluation the trajectory leaves the training distribution — not because CUDA words are rare, but because compiler diagnostics and self-correction text are. RL on such trajectories has to learn from a state distribution it has not seen; entropy rises; the ratio becomes unstable.

This also explains why RFT rescues the run. Rejection fine-tuning keeps only trajectories that succeeded. Successful trajectories contain little debug prose — our `after_success` sample has 12,147 tokens against 474,728 for failures. Anchoring the policy on them removes precisely the region where the probability cost lives.

If this reading is right, then "feed more CUDA data" targets an object we measured at probability 1.0, while the actionable lever is what the environment says after failure and how the policy is asked to recover.

A competing account we cannot rule out. The prose may be a *symptom* of confusion rather than a *source* of instability: a model that is lost writes uncertain text *because* it is lost, and the epistemic state is what destabilises training. Under that reading, prose cost is a measurement of confusion, and an intervention targeting prose would suppress the symptom while leaving the state intact. Our RFT argument does not distinguish these — RFT removes unsuccessful trajectories, not prose.

Two experiments would separate them, and neither is run here. First, keep the content of a failure message but change its formatting (a compiler with a different diagnostic style): the prose distribution should shift while the code should not. Second, ask for a rewrite after a *success*: that produces prose of similar length without the diagnostic content, separating "more prose" from "bad news."

Confirming the account at all requires logging entropy and token probabilities through a training run, which we cannot do.

5.5 A methodological failure, kept because it explains the design

Our first pass, using a local 4B model, produced a headline of 10.00% for `triton_specific` at $p = 0.0156$. It was wrong. The classifier deliberately excluded `load`/`zeros`/`store` from the prose-side markers to avoid mislabelling English sentences, which meant only occurrences *inside code segments* could be labelled Triton-specific. That subgroup held 20 of 451 total occurrences — a 96% undercount — and happened to include two extreme samples. The same word, split by the typing rule:

```
load labelled NL_reasoning      n=6   mean -0.044   lowprob  0.00%
load labelled triton_specific   n=4   mean -1.034   lowprob 25.00%
```

The between-group difference was manufactured by the classification rule. Recomputing over all 451 occurrences moved p from 0.0156 to 0.211.

Two rules came out of this and govern the present paper: **class statistics are computed over every token, never over a pre-partitioned subset**, and **every headline is checked under a second definition** so that a result holding under only one boundary is visible as such.

A third rule came later: **aggregate rates are not enough.** The 816-token and 189-token readings in §4.7 exist because rates cannot distinguish "the model was choosing an implementation" from "the model was choosing a word," and that distinction is the entire question.

5.6 A second methodological failure: the driver

Family B's success rate of 17.5% is not a model property. Of 32 failures, 16 were `NameError` caused by the driver accepting **any** fenced block as the final candidate: in 14 of 39 runs the model's last output was a fragment (63–239 characters, e.g. `group_id = pid // (GROUP_M * num_pid_n)`) rather than a complete file, and the driver ran the fragment. Four more failed on out-of-memory from a task whose peak we mis-estimated. Nine were genuine output mismatches.

Excluding the fragment runs, the success rate on complete files is 28% (7/25). This does not change the token-level conclusions — fragments are still code tokens in the same positions — but it does inflate the share of `after_failure` in the corpus.

---

6. Limitations

1. **Model state.** The measurement is on an aligned model at inference. The base-model control (§5.2) is missing. Everything here is conditional on that difference, which is why the title carries it.
2. **Single model family.** All API runs use one model; the 4B replication in Family A used a different family but a weaker stack.
3. **Task scope — and the kind of fusion tested.** The distinction that matters is not "reduction versus chain" but "**chain with a trivial organisational choice versus chain with a non-obvious one**." All five Family B tasks have the trivial kind: a heavy op followed by elementwise passes, where the fusion decision is "fuse the tail into the epilogue or do a second pass," and both options are easy to write once the convolution is correct. The genuinely hard organisational decisions — a tiling that keeps an intermediate in shared memory, a transpose that changes the access pattern, split-K versus two-pass reduction for a reduction

that does not fit one block, FlashAttention-style online softmax — are **not represented at all**. The null result on RQ3 is therefore a null result about the easy kind of fusion.

4. **Selection effect in Family B.** Success rate 17.5%; `after_failure` holds 87.5% of tokens. The code-side density of 0.60% is measured mostly in the context of a struggling model. Whether it differs when the model is succeeding is not answerable from this corpus.
5. **Statistical dependence.** Tokens within a completion are not independent, so exact tests are anti-conservative (§3.7). No cluster-level analysis is reported.
6. **No training.** We do not measure whether low probability causes importance-ratio explosion, nor the relative contribution of precision mismatch.
7. **One collection path.** A single driver against a single endpoint; no cross-engine replication.
8. **Region assignment by fence state.** A token is `code` or `prose` by fenced-block parity. This misclassifies fenced blocks used for non-code content (we found a few), and comments inside code blocks are counted as code — which biases the code-side rate *upward*, not downward.
9. **An unseparated alternative for the context effect.** `after_failure` may reflect *responding to bad news* or simply *writing new content* (after a failure the model rewrites; after a success it refines). Truncating the message does not distinguish these. Separating them requires a condition that asks for a rewrite after a success.

---

7. Conclusion

Whatever the reported 10 measures, it is not a property of the domain's vocabulary. Over 98,533 tokens on reduction tasks and 542,611 on operator chains, CUDA/Triton vocabulary carries *less* low-probability mass than the surrounding text; inside code fences the rate is 0.33% once comments are removed, outside 13.56%, a 41× difference; the worst 5% of code tokens carry 98.1% of the code's cost against 30.3% for prose; and the absolute minimum across 322,375 code tokens is 2.2×10 , four hundred times above the claimed floor.

Because alignment cannot alter corpus statistics, a rarity-driven floor would have survived it. It did not. That leaves the causal chain's attribution without its stated mechanism.

What the data support is that the cost tracks the kind of text being written, and that composed text concentrates where the model is failing. Whether that makes the prose a cause of instability or a symptom of it is open, and the experiments that would decide it are named in §5.4.

References

("CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation" 2026) CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation. arXiv:2602.24286.

("CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation" 2026) CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation. Technical blog, 2026-03.

("DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations" 2026) DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations. arXiv:2602.05885.

(Liu and others 2025) Liu et al. Your Efficient RL Framework Secretly Brings You Off-Policy RL Training.

("Jet-RL: Precision Mismatch in Reinforcement Learning Training" 2026) Jet-RL: precision mismatch in RL training.

("Precision Mismatch in Reasoning Models" 2026) Precision Mismatch in Reasoning Models.

("First-Order Approximation for Stable LLM-RL Training" 2026) First-Order Approximation for Stable LLM-RL Training.

(Carlini and others 2024) Carlini et al. Stealing Part of a Production Language Model.

("How Open Must Language Models Be to Enable Reliable Scientific Inference?" 2026) How Open Must Language Models be to Enable Reliable Scientific Inference? (2026).

("Surrogate Fidelity: When Can Open LLMs Explain Closed Ones?" 2026) Surrogate Fidelity: When Can Open LLMs Explain Closed Ones? (Meta).

("Log Probability Tracking of LLM APIs" 2026) alphaXiv overview of arXiv:2602.24286 (independent reading; notes the entropy observation and that the association does not establish a single cause).

("KernelBench: Can LLMs Write Efficient GPU Kernels?" 2025) KernelBench: Can LLMs Write Efficient GPU Kernels? (level 2 operator chains).

Appendix A — Reproducibility

```
# Pre-flight
python scripts/check_plaintext_logprobs.py --model deepseek-chat --max-tokens 16000

# Family A
python scripts/loop_driver.py --tasks row_reduce vector_affine \
    --conditions full truncated --repeats 16 --max-turns 8 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-formal
python scripts/loop_driver.py --heldout --tasks layernorm_rows softmax_rows \
    --conditions full truncated --repeats 16 --max-turns 8 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-formal

# Family B
python scripts/make_kb2_tasks.py --out tasks-kb2 --names \
    12_Gemm_Multiply_LeakyReLU 1_Conv2D_ReLU_BiasAdd 14_Gemm_Divide_Sum_Scaling \
    17_Conv2d_InstanceNorm_Divide 19_ConvTranspose2d_GELU_GroupNorm
python scripts/loop_driver.py --tasks-root tasks-kb2 \
    --tasks 12_gemm_multiply_leakyrelu 1_conv2d_relu_biasadd \
            14_gemm_divide_sum_scaling 17_conv2d_instancenorm_divide \
            19_convtranspose2d_gelu_groupnorm \
    --conditions full truncated --repeats 4 --max-turns 6 \
    --model deepseek-chat --runs-root lowprob-attribution/runs-kb2

# Analysis
python scripts/stream_stats.py lowprob-attribution/runs-kb2
python scripts/make_figures.py
```

Appendix B — Production record

Produced with an AI assistant. `PROCESS.md` records the collaboration history, including the withdrawn first pass and the typing bias behind it. The v2 collections were built after a review pointed out that v1 had only measured reduction-shaped tasks; the KernelBench level-2 directory had been present locally throughout and had never been opened.

Carlini, Nicholas, and others. 2024. "Stealing Part of a Production Language Model." In *International Conference on Machine Learning*.

"CUDA Agent: Agentic RL for High-Performance CUDA Kernel Generation." 2026. Technical blog.

"CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation." 2026. https://arxiv.org/abs/2602.24286.

"DR. KERNEL: Reinforcement Learning Done Right for Triton Kernel Generations." 2026. https://arxiv.org/abs/2602.05885.

"First-Order Approximation for Stable LLM-RL Training." 2026.

"How Open Must Language Models Be to Enable Reliable Scientific Inference?" 2026.

"Jet-RL: Precision Mismatch in Reinforcement Learning Training." 2026.

"KernelBench: Can LLMs Write Efficient GPU Kernels?" 2025. https://arxiv.org/abs/2501.11067.

Liu, and others. 2025. "Your Efficient RL Framework Secretly Brings You Off-Policy RL Training."

"Log Probability Tracking of LLM APIs." 2026.

"Precision Mismatch in Reasoning Models." 2026.

"Surrogate Fidelity: When Can Open LLMs Explain Closed Ones?" 2026.